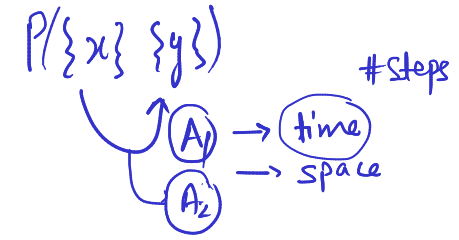


Computational Complexity



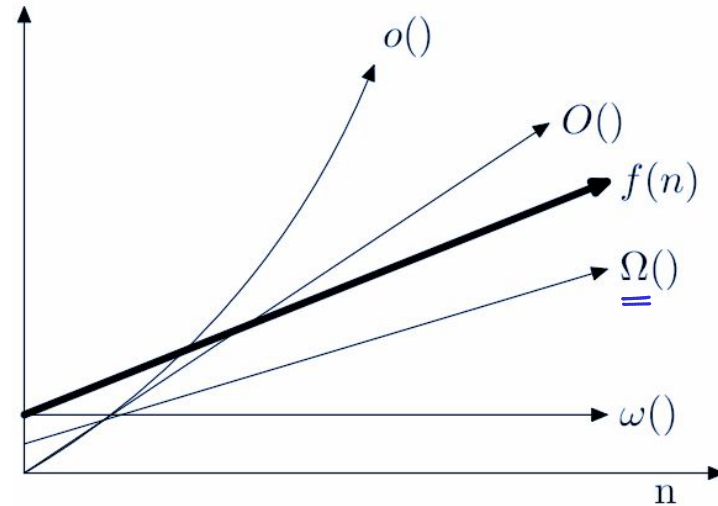
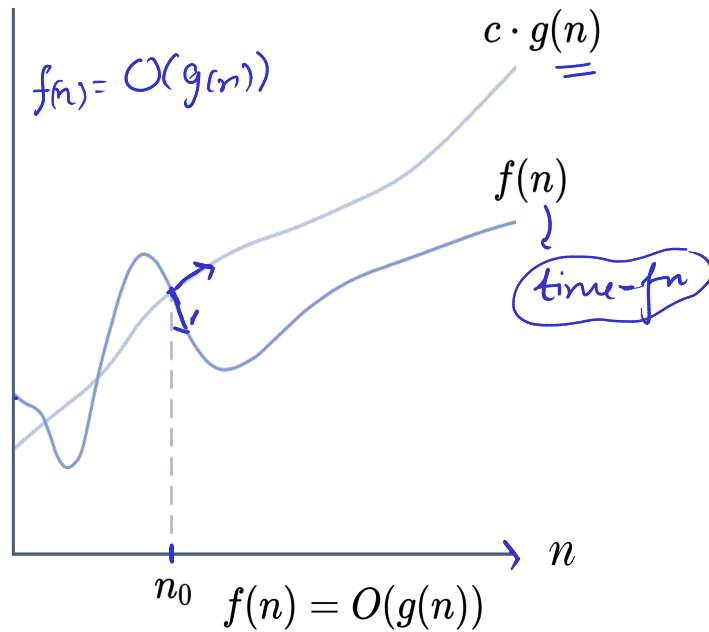
- **Computational Complexity of a Problem**

- A mathematical representation of computational resources required
 - to execute an **algorithm** that solves the corresponding problem
 - a problem can be solved by different algorithms of different complexities

$$O(g) := \{f : \mathbb{N} \rightarrow \mathbb{R}_+ : \exists c \in \mathbb{R} > 0, n_0 \in \mathbb{N} : \forall n \in \mathbb{N}, n \geq n_0 : f(n) \leq c \cdot g(n)\}$$

$1, 2, \dots$

$O(n)$
size



Handwritten notes: $f(n) > c \cdot g(n)$, $\Theta(f(n))$, and a wavy line representing $f(n)$.

Computational Complexity

- Complexity of unsorted database search?

- $O(n)$ in the worst case!

→ quantum algorithm $O(\sqrt{n})$ *quadratic speedup*
Grover's

$f(n) = \Theta(1)$ **constant**

binary search $f(n) = O(\log n)$ **logarithmic**

$f(n) = O(\log^k(n))$ for a $k \in \mathbb{N}$ **polylogarithmic**

unsorted DB search $f(n) = O(n)$ **linear**

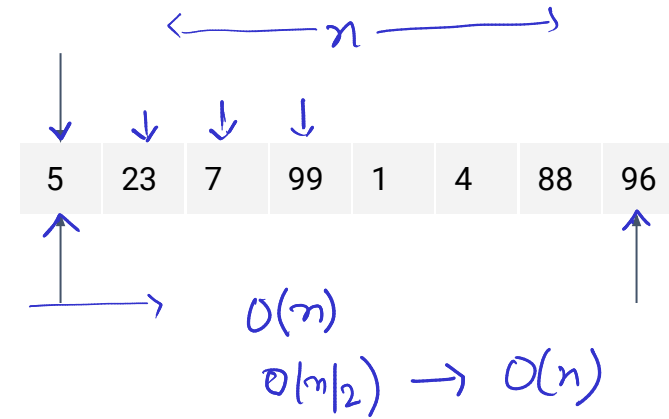
$f(n) = O(n^2)$ **quadratic**

$f(n) = O(n^k)$, for a $k \in \mathbb{N}$ **polynomial**

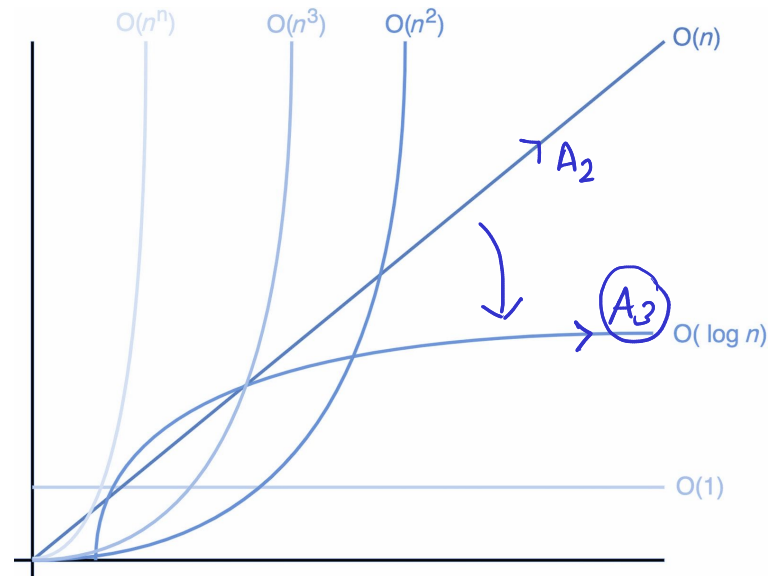
$f(n) = \omega(n^k), \forall k \in \mathbb{N}$ **superpolynomial**

$f(n) = o(2^{cn}), \forall 0 < c \in \mathbb{R}$ **subexponential**

→ $f(n) = \underline{O(2^{cn})}$ for a $0 < c \in \mathbb{R}$ **exponential**



④



A_1
 A_2
 A_3